PARSING THE INPUT FILE

I wanted to break the input file into tokens and these would all be Strings. I wanted to replicate how Parameters are passed to a C program.

Int main (int argc, Const char ** argv)

Now argv has the parameters in an array, of pointers to Const chars. I wanted to have a NULL Value in the last element, because this would be useful in debugging.

Using input file as this: (where ▽ is Space, ascii 0x20)

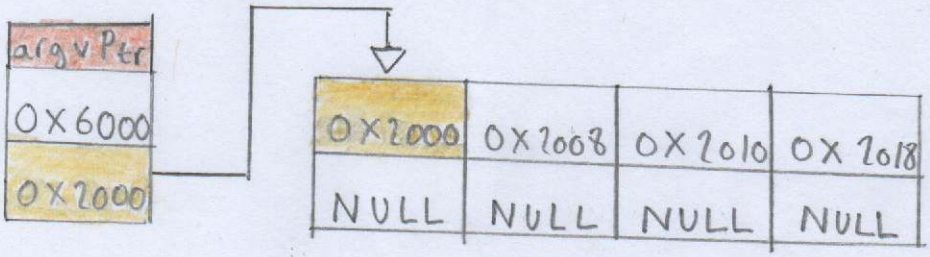
" ▽▽▽ cat ▽\n ▽▽ and ▽ \n \n dog \t \t \n"

By seeing how many char bytes have a delim next to them, we can malloc this many bytes X 8. This will be the maximum # of Strings, but we may have Something like: "how ▽ are ▽ you" in quotes. Then this is 1 String and not 3. I called this Variable argv Len. So I put this max # of words on a register and malloc it.

```
mov     rdi, [argv Len]
Shl     rdi, 3 ; need to store addresses, so X8
Call    malloc
mov     [argv Ptr], rax
```

argv Ptr will now point to the heap and become like argv in main.

Then I wanted NULL in every 8 bytes.



Now to move through the data, we can use a register as a *pointer.

```
mov    r12, [argvPtr]
```

Now we can get the address by: [r12 + r8 * 8]
where we use r8 as an index 0...n.

I need to get this input file and place it in the heap. I could have stored the pointer (argvPtr) to this as a local stack variable, but I decided to place it in the .bss section as it is easier for all functions to access.

Before we did the above steps, we had to grab the input file into memory.

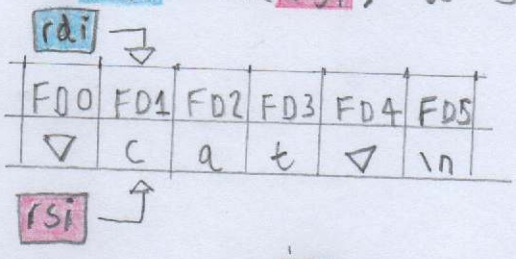
1) use system call #8 to find out how many bytes are in the file
2) now I allocated this many bytes +1 for '\0' on heap with malloc
3) put the heap address in a resq in .bss (heapAllocFile)

Now I get rid of delimetres from our input file, so in the .data section, I declared an array:

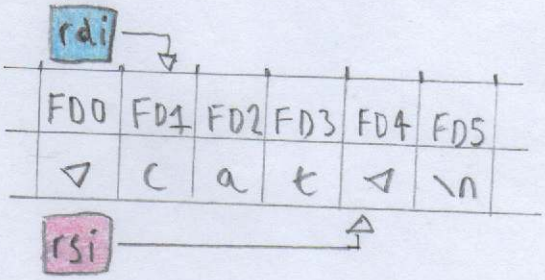
```
delims:   resb   0x09, 0x0a, 0x20, 0x00
```

I used this array in a function that returns true/false on register rax if a byte matches any of these values. In this function:

1) I set up 2 pointers (rdi and rsi) to start on a non-delimetre



2) Increment rsi one byte at a time until we find a delimeter



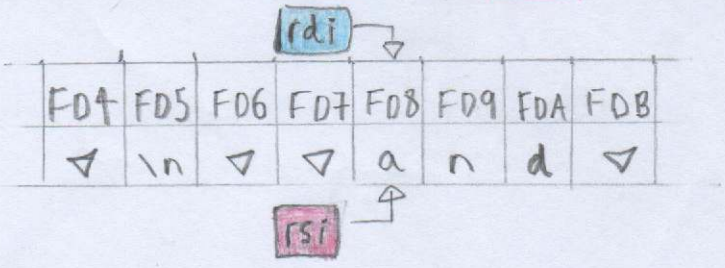
3) Subtract address of rdi from rsi and then +1 for a string term '\0'

4) malloc this amount on the heap

5) Using my function COPY_BYTES we can copy from the source to the destination 1 byte at a time until we reach the # of bytes we need to copy.

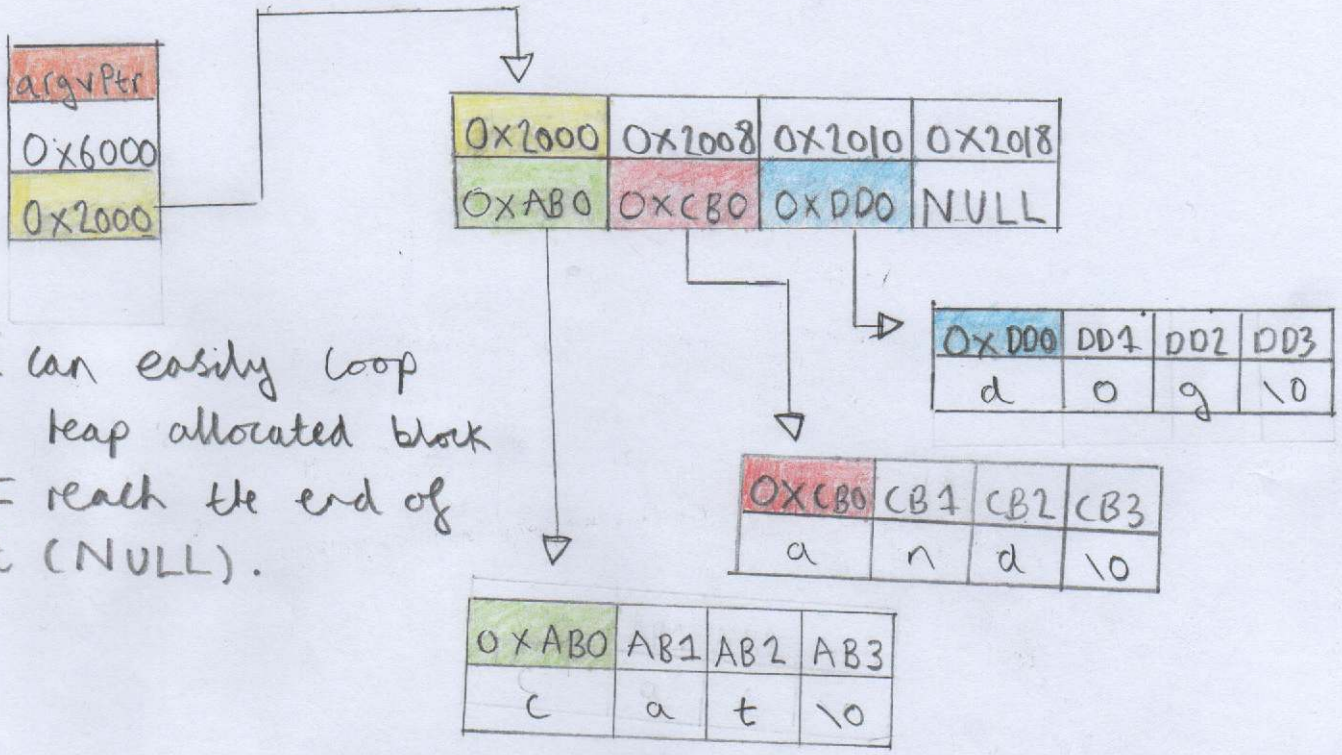
6) add '\0' byte at the end of the string and put this heap address in the argvPtr array.

7) move both rdi and rsi on the next non-delimeter.



8) Repeat these steps from #2

Now I have a pointer to the heap and each resq (8 bytes) holds a pointer to a char array on the heap.



Now I can easily loop over the heap allocated block until I reach the end of the list (NULL).

```
Xor     rbx, rbx    ; used to check NULL
Xor     r8, r8      ; used as an index 0...n
Mov     r12, [argvPtr]

.looping:
    cmp     [r12 + r8 * 0x08], rbx ; NULL?
    je      .ended

    Mov     rdi, [r12 + r8 * 0x08]
    Call    printStrings

    inc     r8
    jmp     .looping
```

This is an example of accessing all the strings and sending them to a function using rdi.